

Pré-processando os dados para o KNN

Olá! No vídeo anterior, executamos o KNN em um certo conjunto de dados de frutas. Este conjunto de dados possui algumas características e escalas diferentes. E como trabalhado no vídeo anterior o KNN pode ter um desempenho ruim caso os atributos ou as colunas não estejam na mesma escala. Então, para acabar com este problema, iremos utilizar o pré-processamento com Min e Max para tentar melhorar o resultado. Primeiro, iremos importar o Min e Max: "from sklearn.preprocessing import MinMaxScaler".

Executando esta célula, teremos importado a escala do Min e Max. Então, para efetuar

a transformação de escala, executaremos a seguinte variável: `mm_MinMaxScaler()`. Assim, iremos ter o Min e Max implementado.

Executaremos separadamente os dados de teste a transformação de escala. Iremos efetuar a seguinte linha: "X_train", utilizando os dados que foram lidos e separados em treino e teste no vídeo anterior, ficando: `X_train = mm.fit_transform(X_train)`.

O que este comando está fazendo?

Ele está pegando os dados de treino e transformando cada uma de suas colunas em dados entre 0 e 1 e já transformando esses dados do conjunto de treino e jogando-os na própria variável no conjunto de treino. Executando este comando e imprimindo a variável "X_train" teremos dados entre 0 e 1. Devemos fazer algo parecido também para o conjunto de teste.

Então, utilizaremos: `X_test = mm.transform(X_test)`.

Na linha anterior, utilizamos: "mm.fit_transform". Nessa linha, estamos utilizando apenas "mm.transform". Por que essa diferença? Nesta segunda linha, estamos apenas transformando os dados de acordo com a escala efetuada nos dados de treino, pois os dados de treino serão utilizados para preparar e construir o modelo. Então, toda transformação deve ser baseada na transformação executada nos dados de treino.

Por isso, utilizaremos o "transform", e não o "fit.transform" para construir uma nova transformação em cima dos dados de teste. Logo, ao executar esta linha e imprimirmos o "X_test", teremos dados que podem passar de 1. No exemplo que estou usando, temos, como último dado o número, 1.0270. Ou seja, ele está maior que 1, mas ele ainda continua em uma escala pequena não será prejudicial. Além do mais, estamos transformando de acordo com os dados de treino. Isso não será prejudicial para a nossa avaliação.

Então, iremos retreinar o KNN. E a linha toda fica: "`knn = KNeighborsClassifier(n_neighbors = 3)`". Dessa maneira, estamos instanciando novamente o método KNN para classificação, que irá procurar apenas os três vizinhos mais próximos para efetuar a classificação.

Utilizando novamente: "`knn.fit(X_train,`", que agora são as variáveis na escala entre 0 e 1

`y_train)`". Então, a linha para treinamento dos dados ficou: "`knn.fit(X_train, y_train)`".

Para constar, não precisamos transformar também os rótulos, basta transformar os atributos. Afinal, só eles que passarão pela técnica de distância. Ao executar esta linha,

teremos retreinado o KNN.

Na linha seguinte: `knn.score(X_test, y_test)`. E o resultado agora foi igual a 1. Ou seja, 100% de acerto. Ao comparar os dados de teste e os seus rótulos ao resultado desses dados de teste no modelo, tivemos 100% de acerto. O que significa que o modelo conseguiu prever perfeitamente a classe de cada um dos dados de teste.

Essa função `score` é capaz de efetuar a predição de acordo com os atributos de teste e comparar o resultado com os rótulos de testes que estão sendo passados.

Dessa maneira, tivemos 100% de acerto.

Se quisermos prever apenas os nossos resultados, podemos utilizar: `knn.predict(X_test)`.

E esta função irá retornar para os dados de teste a classe que eles devem estar de acordo com o KNN.

E podemos compará-lo com os rótulos de teste "`y_test`".

Ao executar esta linha, podemos comparar cada um dos elementos e verificar o que foi acerto e erro. Ao utilizar a função `score` essa comparação já é feita automaticamente e já é retornada a probabilidade de acerto.

Com isso, podemos perceber que o nosso `score` que antes era de 0,6 ou 60% de acerto tornou-se 100% de acerto apenas com a modificação da escala dos dados, ou seja, mudando a escala dos dados a comparação se tornou melhor uma vez que a escala deles não fez diferença no cálculo da distância. Outra variação que pode ser executada é a alteração no número de vizinhos pesquisados.

Essa alteração pode dar resultados diferentes.

Eu aconselho vocês alterarem a quantidade de vizinhos e verificarem como o modelo irá se comportar. No próximo vídeo, iremos mostrar o KNN para regressão.

Até lá!